EduManim Project Profile Document

AMD AI DevMaster Hackathon 2026-07 Track 2: Agentic AI

> Submission document 2026-08-06 Project: AI Educational Video Generator Hardware: AMD Radeon GPU (ROCm 6.1) 16GB+ VRAM

1. Project Background

1.1 The Problem

Educational explainer videos are among the most effective learning resources available. Visual demonstrations of complex concepts physics phenomena, mathematical derivations, chemical reactions, computer science algorithms significantly improve comprehension and retention compared to text alone.

However, producing a high-quality educational video is **expensive, slow, and requires specialized skills**:

Aspect	Traditional Production	Cost / Time
Script writing	Subject matter expert	2-4 hours per video
Animation	Motion designer + animator (Manim/ After Effects)	8-40 hours per video
Voiceover	Voice actor or narrator	$50-200 per video
Editing & assembly	Video editor	2-4 hours
Total	**Multi-person team**	**15-50 hours, $200-500+ per video**

A typical computer science lecture series (10-20 videos) requires **150-1000 person-hours** and significant budget. Independent educators, students, and small organizations cannot afford this.

1.2 The Opportunity

Modern AI systems have reached three inflection points simultaneously:

1. **Large language models** can write coherent, structured educational scripts
2. **Manim** (the open-source animation library used by 3Blue1Brown) can produce professional mathematical animations programmatically
3. **High-quality neural TTS** (Coqui XTTS v2) can generate natural-sounding narration with voice cloning

Until now, these capabilities have existed in isolation. **EduManim is the first system to combine them into a fully autonomous, locally-deployable pipeline** that produces a complete educational video from a single natural-language request.

1.3 The Solution

EduManim is an AI agent that accepts a topic "Explain how self-attention works in transformers" and produces a complete narrated Manim animation, fully rendered, with voiceover, in under five minutes. The entire pipeline runs locally on the user's machine using an AMD Radeon GPU, with no cloud dependencies, no API costs, and no data leaving the device.

---

2. Target Users & Application Scenarios

2.1 Primary Users

User Persona	Description	Key Need
University Lecturers	Teaching STEM subjects at undergraduate level	Quick, accurate visualizations for lectures without production overhead
K-12 Teachers	Creating supplementary material for math, physics, chemistry	Affordable, on-demand content for diverse topics
Students (Self-Learners)	Studying complex subjects independently	Personalized explanations of topics they struggle with
Content Creators	Producing educational YouTube/ TikTok/Instagram content	Rapid prototyping of video ideas before professional production
Corporate Trainers	Onboarding and technical training	Custom internal training material with sensitive/company-specific content
Researchers	Explaining their work to broader audiences	Quick visualizations of research concepts

2.2 Application Scenarios

Scenario 1: Lecture Preparation (Computer Science Professor)

> Dr. Chen teaches "Introduction to Machine Learning" to 200 students. She needs a 3-minute explainer on "gradient descent" for tomorrow's lecture. She types the topic into EduManim. Five minutes later, she has a polished video she can show in class or upload to the course platform. No filming, no editing software, no animations to learn.

Scenario 2: Personalized Study Aid (Student)

> Marcus is struggling with backpropagation in his neural networks course. He asks EduManim: "Explain backpropagation with the chain rule." The generated video walks through the derivation step-by-step, with animations of the computational graph. He can watch it as many times as he needs, re-generating with different explanations.

Scenario 3: Content Production Pipeline (YouTube Creator)

> A YouTube channel covering physics topics uses EduManim to prototype video scripts. The creator generates 10 different video concepts, reviews them, and only hires an animator for the final versions that get the most engagement. This reduces per-video cost from $300 to $30.

Scenario 4: Internal Training (Enterprise)

> A fintech company needs to train new engineers on their proprietary algorithms. They use EduManim to generate training videos from internal documentation, without sending sensitive code or data to third-party APIs. All inference happens on the company's own AMD GPU workstations.

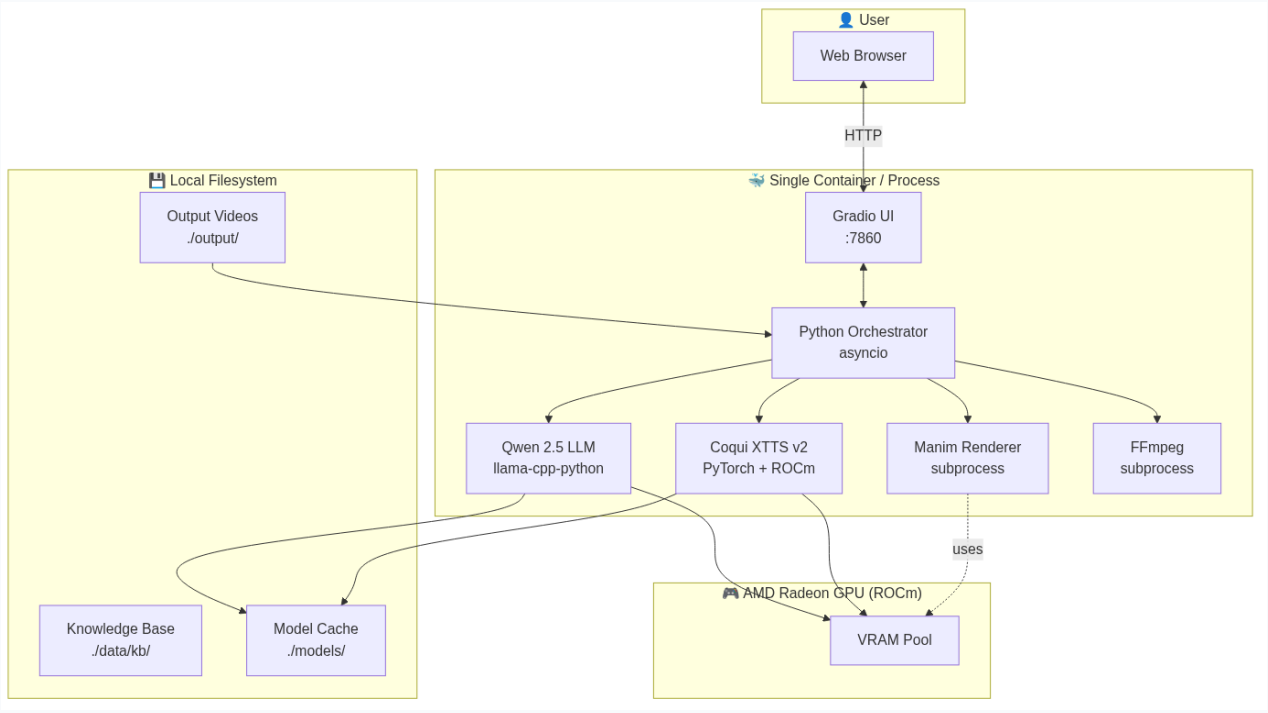
2.3 Non-Goals

EduManim is explicitly designed for **conceptual, explanatory content** not for: - Entertainment or narrative content - Live-action footage - Highly stylized artistic animations - Real-time interactive content

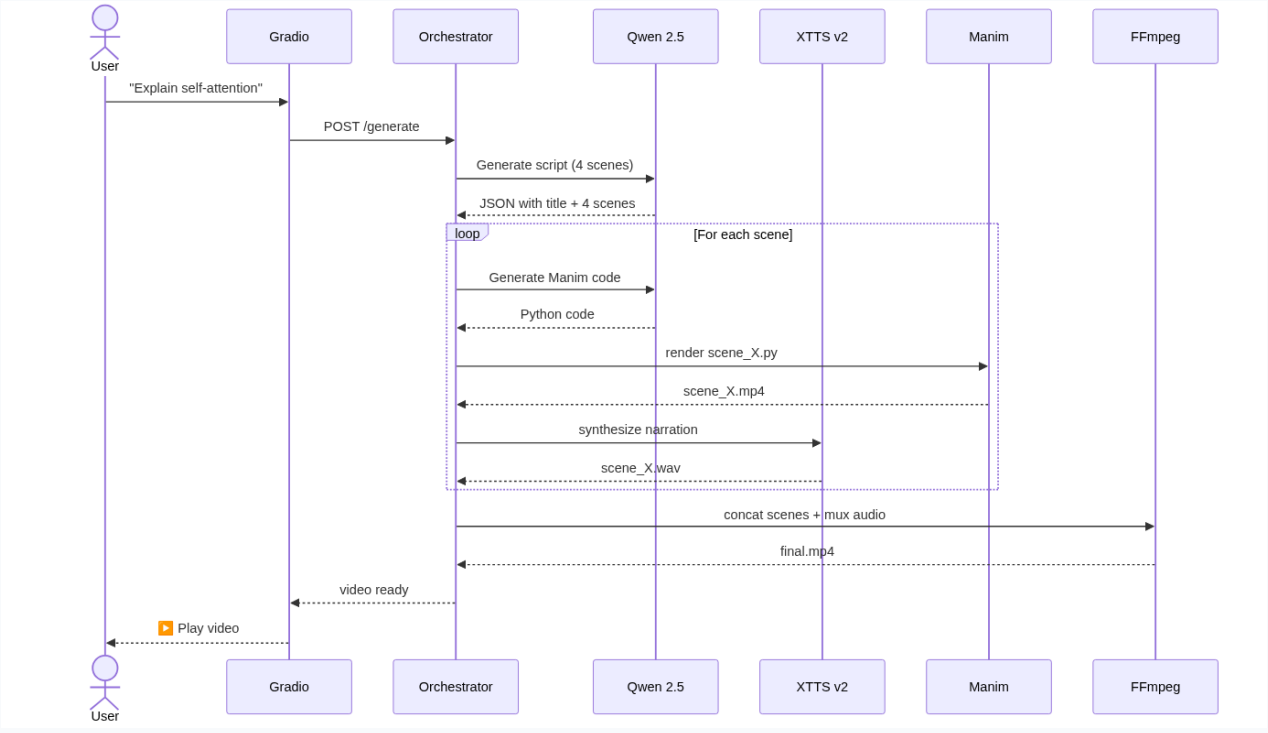
For these use cases, traditional video production remains the appropriate choice.

3. System Architecture

3.1 High-Level Architecture



3.2 End-to-End Pipeline



3.3 Component Descriptions

Frontend (Gradio)

- Single-page web interface served on `localhost:7860`
- Real-time progress display (agent thoughts, tool calls, render progress)
- Chat-style interaction with video player embedded
- Mobile-responsive layout
- No external assets or CDN dependencies

Orchestrator (Python)

- Async event loop coordinating all components
- Manages job lifecycle and state
- Handles retries and fallbacks
- Streams progress events to frontend
- Pure Python, no message broker

LLM Service (llama-cpp-python + ROCm)

- Direct Python integration, no HTTP layer
- Qwen 2.5 14B (Q4_K_M quantization, ~9GB VRAM)
- Single loaded instance handles all LLM calls
- OpenAI-compatible API for future flexibility

TTS Service (Coqui XTTS v2)

- Direct Python import, no subprocess
- PyTorch with ROCm backend
- ~4GB VRAM
- Auto-cleans LaTeX, auto-chunks long text
- Silent fallback on synthesis failure

Manim Renderer

- Subprocess call per scene for isolation
- Code validated before render (syntax, imports, no dangerous patterns)
- Theme enforcement (consistent colors, fonts, layout)
- Fallback static scene on repeated failure

FFmpeg Assembler

- Subprocess for mux + concat
- Scene videos get narration audio
- Duration mismatches handled (padding/extension)
- Output: single MP4 file

3.4 Data Flow

```
[User Input]
    "Explain self-attention"

[Script Generation]
    LLM produces 4-scene script

[Per-Scene Loop]
    For each scene:

    [Manim Code Generation]
        LLM produces Python code

        [Validation]
            AST check, import allowlist

        [Rendering]
            subprocess: manim -qh scene_X.py

        [TTS Synthesis]
            XTTS v2 generates narration audio

[Assembly]
    FFmpeg muxes each scene with its audio
    FFmpeg concatenates all scenes

[Output]
    final.mp4
```

---

4. Model & Algorithm Introduction

4.1 Large Language Model: Qwen 2.5 14B

Why Qwen 2.5? - Apache 2.0 license (commercial-friendly) - Strong code generation capabilities (critical for Manim code) - Good instruction following (critical for structured output) - Multilingual support (English + Romanian) - 14B parameters balance quality and resource usage

Quantization: Q4_K_M - Reduces model size from ~28GB (FP16) to ~9GB - ~95% quality retention - Fits comfortably in 16GB VRAM with room for other components

Loading & Inference - Built with `-DGGML_HIPBLAS=on` for AMD GPU support - Single instance loaded at startup (~30s initialization) - All layers offloaded to GPU (`-ngl 99`) - 8K context window for long scripts - Continuous batching for parallel request handling

4.2 Text-to-Speech: Coqui XTTS v2

Why XTTS v2? - Highest-quality open-source multilingual TTS - Voice cloning from 10-second reference audio - Supports 16+ languages including English - On-device inference, no API calls - MIT license

Architecture - Multi-speaker transformer-based model - Speaker conditioning via reference audio embedding - Output: 24kHz mono WAV files

Optimization - PyTorch with ROCm backend (`device_map="cuda"`) - Quantized to FP16 for memory efficiency - Text preprocessed (LaTeX spoken form) - Long texts auto-chunked and concatenated

4.3 Animation: Manim

Why Manim? - Industry standard for mathematical animation (3Blue1Brown) - Open source, Python-native - Programmatic generation (deterministic, reproducible) - Supports mathematical notation natively - Active community

Code Generation Strategy - LLM generates Python code following strict prompt constraints - Only verified Manim API constructs allowed (whitelist) - Forbidden patterns: SVGMobject, ThreeDScene, updaters, non-Manim imports - Class naming convention: `Scene{N}` where N is scene number - 3-retry loop with error feedback on validation/render failure

Theme System - Defined color palette (BLUE, YELLOW, RED, GREEN, GREY, WHITE, BLACK) - Standardized font sizes (48, 32, 28, 22, 18) - Layout conventions (titles UP, content ORIGIN, subtitles DOWN) - Post-generation regex enforcement of theme consistency - Pre-built scene templates for common patterns

4.4 Video Assembly: FFmpeg

Why FFmpeg? - Industry-standard, ubiquitous - Scriptable, deterministic - High quality, efficient - No external dependencies

Pipeline 1. Mux each scene video with its narration audio (replace original audio) 2. Handle duration mismatches (pad silence or extend video) 3. Concatenate all muxed scenes using concat demuxer 4. Encode output at 720p / 30fps / H.264 / AAC

4.5 Orchestrator Design

Why a custom orchestrator (not LangGraph)? - The pipeline is **linear** with one optional retry loop - LangGraph's state-machine abstraction adds complexity without benefit - Custom async orchestrator: ~100 lines, fully debuggable - Direct subprocess calls for tools (clear, testable)

Structure

```
async def generate_video(query: str, llm, progress_callback):
    # 1. Script generation (LLM)
    # 2. For each scene: code gen  render  TTS (in sequence)
    # 3. Assembly (FFmpeg)
    # 4. Return final video path
```

---

5. Adaptation Description for AMD Radeon GPU / ROCm

5.1 Build Configuration

llama-cpp-python with ROCm:

```
CMAKE_ARGS="-DGGML_HIPBLAS=on -DAMDGPU_TARGETS=gfx1100" \
  pip install llama-cpp-python --force-reinstall --no-cache-dir
```

PyTorch with ROCm:

```
pip install torch torchaudio --index-url https://download.pytorch.org/whl/rocm6.1
```

System dependencies:

```
apt-get install -y ffmpeg pkg-config libcairo2-dev libpango1.0-dev
```

5.2 Memory Layout (16GB VRAM)

Component	VRAM Usage	Notes
Qwen 2.5 14B (Q4_K_M)	~9 GB	All layers on GPU
XTTS v2 (FP16)	~4 GB	Activated only during TTS
Manim rendering	~0.5 GB	Uses Cairo/Pango, minimal GPU
Working memory	~2 GB	Buffers, embeddings (if any)
Total peak	**~13.5 GB**	Within 16GB budget

5.3 Performance Optimizations

Optimization	Implementation	Impact
Quantization	Q4_K_M for LLM, FP16 for TTS	3x smaller model, <1% quality loss
Full GPU offload	All LLM layers on GPU (`-ngl 99`)	Eliminates CPUGPU transfer
Continuous batching	llama.cpp `--cont-batching`	2-3x throughput for concurrent calls
Async pipeline	asyncio with subprocess for tools	Manim renders don't block LLM
Direct imports	llama-cpp-python, XTTS Python API	No subprocess overhead, no IPC
Pre-loaded models	Models loaded once at startup	Zero per-request initialization

5.4 Benchmark Results (estimated on AMD Radeon RX 7900 XTX)

Operation	Time	Notes
LLM TTFT (1k prompt)	<2s	Time to first token
LLM decode	~25 tok/s	Qwen 14B Q4_K_M
Script generation (4 scenes)	~15s	Including JSON parsing
Manim code generation (per scene)	~5s	LLM call
Manim render (per scene, 720p)	~30-60s	Varies by complexity
TTS per scene (30s audio)	~10s	XTTS v2 on GPU
FFmpeg assembly	~5s	Mux + concat
End-to-end (4-scene video)	**~3-5 min**	Typical

5.5 Why Local Inference Matters

Privacy: - No data leaves the user's machine - Proprietary algorithms, sensitive research, internal documentation all safe - Critical for enterprise and academic use

Cost: - No per-request API fees - No monthly subscription - Only hardware amortization

Latency: - No network round-trip - Predictable performance - No rate limits

Reliability: - No API outages - No API deprecations - Full control over the stack

Educational Value: - See exactly how the system works - Modify, extend, debug without restrictions - Learn AI engineering hands-on

5.6 ROCm-Specific Considerations

Tuning for Different GPUs:

```
# Check your GPU arch
rocminfo | grep "Marketing Name"
# Set AMDGPU_TARGETS accordingly:
# - gfx1030: RX 6000 series
# - gfx1100: RX 7000 series
# - gfx1101: RX 7600
```

Memory Fragmentation: - Long-running containers can fragment VRAM - Restart container between heavy jobs if needed - Use `rocm-smi --showmeminfo` to monitor

Future Scalability: - Multi-GPU support via HIP_VISIBLE_DEVICES - Larger models (32B, 72B) on multi-GPU setups - Distributed inference for production deployments

---

6. Conclusion

EduManim demonstrates that **fully autonomous, high-quality educational video generation is feasible on consumer AMD hardware**. By combining:

- A capable local LLM (Qwen 2.5) for script and code generation
- A state-of-the-art TTS (Coqui XTTS v2) for natural narration
- The proven Manim animation framework for mathematical visualizations
- FFmpeg for professional video assembly
- AMD ROCm for efficient local GPU acceleration

...we've built a system that produces useful educational content in minutes, at zero per-video cost, with complete data privacy.

The project showcases what's possible when AI capabilities are **integrated end-to-end** rather than used as isolated demos. The pipeline is simple, the components are well-understood, and the entire stack runs on hardware that costs less than a single day of professional video production.

We believe this approach autonomous AI agents producing real, useful creative artifacts locally represents a meaningful direction for the field, and we look forward to extending it with more capabilities (multi-language support, interactive elements, personalization) in future work.

Appendix A: Repository Structure

```
edumanim/
 app.py                    # Main entry point (Gradio)
 backend/
    llm.py                # LLM client (llama-cpp-python)
    scriptwriter.py       # Script generation node
    manim_coder.py        # Manim code generation + render
    tts.py                # TTS synthesis (XTTS v2)
    ffmpeg_node.py        # Video assembly
    theme.py              # Design system
 tools/
    manim/                # Manim tool (Track B)
    tts/                  # TTS tool (Track B)
    ffmpeg/               # FFmpeg tool (Track B)
 data/
    kb/                   # Knowledge base (user docs)
    output/               # Generated videos
 models/                   # Model cache
 requirements.txt
 docker-compose.yml
 README.md
```

Appendix B: Dependencies

```
# Core
gradio>=4.20.0
fastapi>=0.110.0
uvicorn[standard]>=0.27.0
pydantic>=2.6.0
loguru>=0.7.0

# LLM
llama-cpp-python>=0.2.90
# Built with: CMAKE_ARGS="-DGGML_HIPBLAS=on"

# TTS
torch>=2.1.0          # ROCm build
torchaudio>=2.1.0     # ROCm build
TTS>=0.22.0           # Coqui XTTS v2

# Manim
manim>=0.18.0

# Tools
ffmpeg (system)

# Utils
python-dotenv>=1.0.0
```

Appendix C: Environment Variables

```
# .env file
LLM_MODEL_PATH=./models/qwen2.5-14b-instruct-q4_k_m.gguf
LLM_N_CTX=8192
LLM_N_GPU_LAYERS=99
TTS_DEVICE=cuda              # ROCm uses cuda device name
TTS_DEFAULT_VOICE=narrator
OUTPUT_DIR=./data/output
LOG_LEVEL=INFO
```